

AirWatch: An End-to-End Serverless Air Quality Index Prediction System for Karachi, Pakistan

Maryam Sultan
10 Pearls Shine Program
Karachi, Pakistan
June 2026

Dashboard: <https://karachi-aqi.streamlit.app>

Code: <https://github.com/maryamm12sak/aqi-predictor>

DagsHub MLflow: <https://dagshub.com/maryamsultan800/aqi-predictor/experiments>

Abstract—This paper presents AirWatch, a fully serverless, end-to-end machine learning system for predicting the Air Quality Index (AQI) in Karachi, Pakistan up to 72 hours in advance. The system fetches real-time air quality and meteorological data every hour using the Open-Meteo Air Quality API and Open-Meteo Weather API, engineers 19 time-based and derived features, stores them in MongoDB Atlas, trains separate machine learning models for 24h, 48h and 72h horizons every day, and serves predictions through an interactive Streamlit web dashboard.

The best performing model – a Stacking Ensemble of Random Forest, XGBoost, and Gradient Boosting with Ridge Regression as the meta-learner – achieved RMSE = 14.36 AQI units, MAE = 9.39, and $R^2 = 0.800$ on a held-out test set for the 24-hour horizon. The system is trained on 33,547 real hourly records spanning January 2022 to June 2026, making it one of the most comprehensive ML-based AQI forecasting systems built for Karachi using exclusively free-tier cloud services.

Index Terms—Air Quality Index, AQI Prediction, Machine Learning, Ensemble Methods, Serverless Architecture, Karachi, Pakistan, MLflow, MongoDB, GitHub Actions

I. INTRODUCTION

Air pollution is one of the leading environmental health risks worldwide. Karachi, Pakistan’s largest city with a population exceeding 20 million, consistently ranks among the most polluted cities in South Asia. The Air Quality Index (AQI) is a standardized scale ranging from 0 (Good) to 500 (Hazardous) that quantifies overall air quality.

Accurate AQI forecasting enables citizens to plan outdoor activities, allows authorities to issue timely health advisories, and supports policy decisions. However, most existing AQI prediction systems for Pakistani cities either rely on expensive proprietary APIs, require dedicated server infrastructure, or produce inaccurate forecasts due to insufficient training data.

AirWatch addresses these limitations by building a complete, automated, serverless AQI prediction pipeline using **only free-tier cloud services**. The system collects data hourly, retrains models daily, and serves real-time predictions through a publicly accessible dashboard.

A. Problem Statement

Given current air quality conditions (AQI, pollutant concentrations, weather parameters, and temporal features) at time

T , predict the AQI at time $T+24h$, $T+48h$, and $T+72h$ for Karachi.

B. Contributions

- A fully serverless, automated AQI prediction pipeline with **zero infrastructure cost**.
- A comprehensive dataset of 33,547 real hourly AQI and weather records for Karachi (2022–2026).
- Systematic evaluation of six ML models, from linear regression to stacking ensembles.
- A publicly deployed interactive dashboard with real-time forecasts and hazard alerts.
- Open-source implementation available on GitHub.

II. BACKGROUND AND RELATED WORK

A. What is AQI?

The Air Quality Index is a dimensionless scale developed by the US Environmental Protection Agency (EPA) to communicate air quality conditions to the public. AQI is computed from pollutant concentration measurements using piecewise linear functions with defined breakpoints. For PM_{2.5} — the primary pollutant used in this work — the breakpoints range from 0–12 $\mu\text{g}/\text{m}^3$ (Good, AQI 0–50) to above 350.5 $\mu\text{g}/\text{m}^3$ (Hazardous, AQI 401–500).

TABLE I
AQI CATEGORIES AND HEALTH IMPLICATIONS

AQI Range	Category	Health Implication
0–50	Good	Air quality is satisfactory
51–100	Moderate	Acceptable for most people
101–150	Unhealthy for Sensitive Groups	Sensitive groups affected
151–200	Unhealthy	Everyone may experience effects
201–300	Very Unhealthy	Health alert for all groups
301–500	Hazardous	Emergency health conditions

B. What is PM_{2.5}?

PM_{2.5} refers to fine particulate matter with diameter less than 2.5 micrometers. These particles are small enough to penetrate deep into the lungs and bloodstream, causing cardiovascular and respiratory diseases. PM_{2.5} is the primary driver

TABLE II
SYSTEM ARCHITECTURE OVERVIEW

Stage	Component	Technology	Schedule
1	Feature Pipeline	Python + Open-Meteo APIs	Every hour
2	Feature Storage	MongoDB Atlas (M0 free)	Persistent
3	Training Pipeline	Scikit-learn (RF, GBM, Ridge, Voting, Stacking) + XGBoost + MLflow	Every day
4	Web Dashboard	Streamlit + Plotly	Always live

of AQI in Karachi due to vehicle emissions, industrial activity, dust storms, and open waste burning.

C. Machine Learning for AQI Prediction

Recent literature has demonstrated the effectiveness of ensemble ML methods for air quality forecasting. Random Forest and Gradient Boosting models consistently outperform linear models due to their ability to capture non-linear relationships between meteorological conditions and pollutant concentrations. Stacking ensembles that combine multiple base learners have shown further improvements by leveraging the complementary strengths of different algorithms.

III. SYSTEM ARCHITECTURE

AirWatch follows a four-stage serverless pipeline where each stage communicates through **MongoDB Atlas** as the central data store.

Serverless means no dedicated server is needed – GitHub Actions provides free compute for scheduled jobs, MongoDB Atlas free cloud database, and Streamlit Cloud free web hosting.

MongoDB Atlas stores each hourly reading as a JSON-like document with all 19 features. The free tier (512 MB) is sufficient for years of hourly records.

GitHub Actions runs two workflows:

- *Feature pipeline* every hour (0 * * * *)
- *Training pipeline* daily at 2 AM UTC

DagsHub + MLflow tracks every training run: all model metrics (RMSE, MAE, R^2) and hyperparameters are automatically logged, creating a complete history of model performance.

A. Feature Store Access

The MongoDB Atlas feature store contains 33,556 real hourly records (Fig. 1) with all engineered features and three target columns (`target_aqi_24h`, `target_aqi_48h`, `target_aqi_72h`). As the database is credentials-protected, it cannot be shared publicly. However, the entire dataset can be recreated from scratch by running `backfill_openmeteo.py`, which fetches 4.5 years of historical data directly from the Open-Meteo API at no cost.

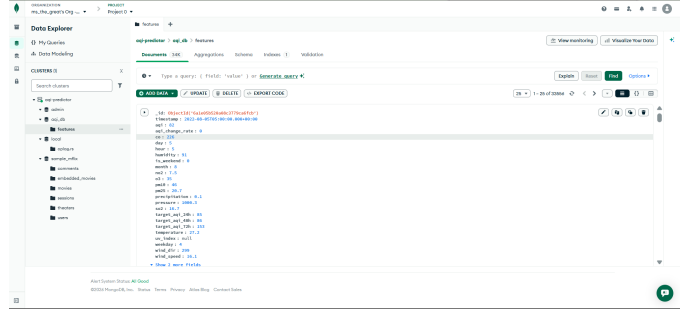


Fig. 1. MongoDB Atlas feature store showing 33,556 hourly records with all features and target columns.

IV. DATA COLLECTION AND FEATURE ENGINEERING

A. Data Sources

All data is obtained from **Open-Meteo** (free, no API key required):

- **Air Quality API** – provides US AQI, PM2.5, PM10, O₃, NO₂, SO₂, CO (CAMS model data).
- **Weather Archive & Forecast APIs** – temperature, humidity, wind speed/direction, precipitation, pressure, UV index (ERA5 reanalysis).

B. Historical Backfill

4.5 years of hourly data (Jan 2022 – June 2026) were fetched for Karachi coordinates (24.8607°N, 67.0011°E). After merging and cleaning, **33,547 valid hourly records** were obtained.

TABLE III
DATASET STATISTICS

Parameter	Value
Date Range	Jan 2022 – June 2026
Valid Records	33,547
AQI Range	14 – 297
Resolution	Hourly

C. Feature Engineering

Nineteen features per record are summarized in Table IV.

D. Target Variables

Separate targets for each horizon:

- `target_aqi_24h` – actual AQI value 24 hours later
- `target_aqi_48h` – actual AQI value 48 hours later
- `target_aqi_72h` – actual AQI value 72 hours later

These are set using **real forward-looking data**: when the feature pipeline runs at time T , it updates the target for the record stored at time $T-24h$ with today's real AQI.

V. EXPLORATORY DATA ANALYSIS

All plots shown below are generated from the actual 33,547 historical records.

TABLE IV
ENGINEERED FEATURES

Feature	Category	Description
hour, day, month, week-day, is_weekend	Temporal	Time components
aqi_change_rate	Derived	Hourly AQI difference (momentum)
wind_u, wind_v	Derived	Wind vector components
temperature, humidity, wind_speed, wind_dir, precipitation	Meteorological	Weather conditions
pm25, pm10, o3, no2, so2, co	Pollutant	Concentrations
aqi	Input/Context	Current AQI (strongest predictor)

A. AQI Distribution

The mean AQI is approximately 91 and the median is 82, both falling in the Moderate category. The dominant category is Moderate (~70.5%), followed by Unhealthy for Sensitive Groups (~20.4%) and Unhealthy (~7.1%). The histogram shows a right-skewed distribution with a secondary peak around AQI 150.

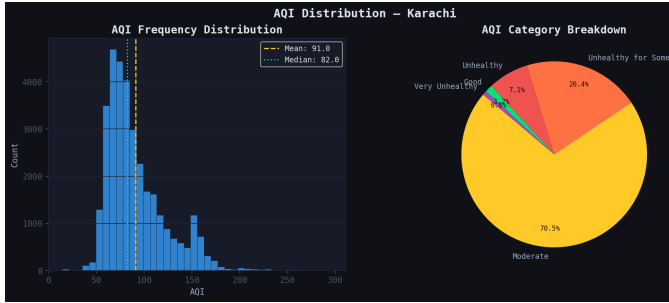


Fig. 2. AQI Frequency Distribution and Category Breakdown – Karachi (33,547 records).

B. Hourly Pattern

Average AQI peaks at 19:00 (~99), just below the Unhealthy threshold of 100, corresponding to evening traffic and reduced atmospheric mixing. Morning hours (00:00–05:00) show the lowest AQI (~89).

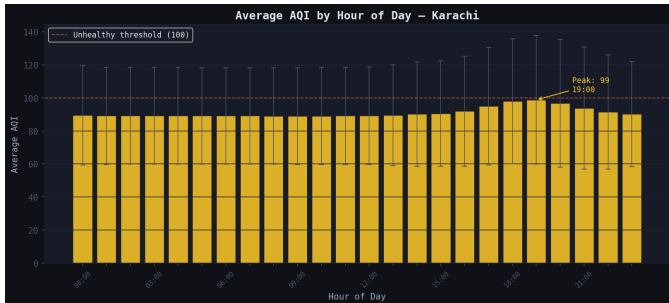


Fig. 3. Average AQI by Hour of Day – Karachi.

C. Seasonal Trends

Monthly average AQI is highest in November–February (~116–101) due to winter temperature inversions. AQI is lowest in April–May (~70–75). The weekday pattern shows minimal variation across days of the week.

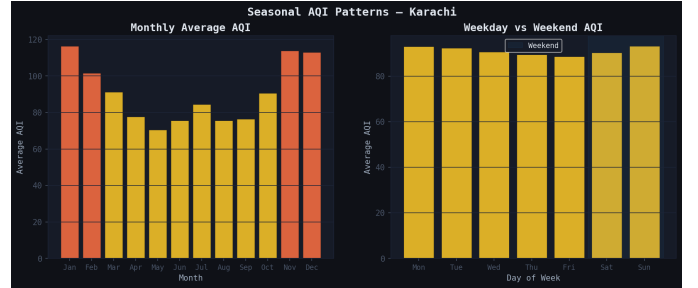


Fig. 4. Seasonal AQI Patterns – Monthly Average and Weekday vs. Weekend.

D. Correlation Heatmap

Strongest correlations with AQI: PM2.5 ($r = 0.72$), SO₂ ($r = 0.56$), CO ($r = 0.53$), NO₂ ($r = 0.45$). Temperature shows a moderate negative correlation ($r = -0.32$). Wind speed is negatively correlated ($r = -0.28$), confirming that higher winds disperse pollutants.

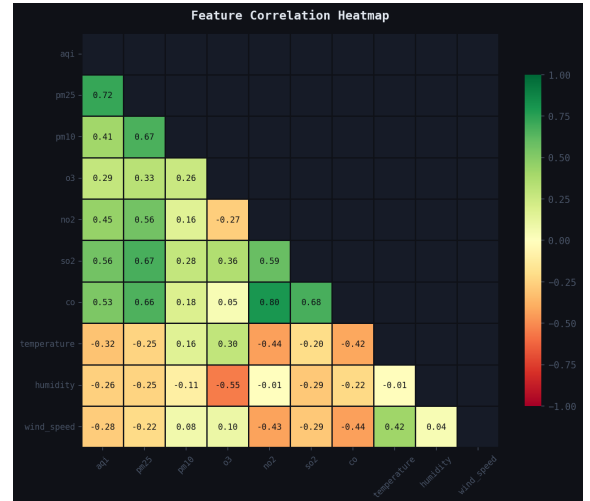


Fig. 5. Feature Correlation Heatmap.

E. Pollutant vs. AQI

PM2.5 shows the strongest relationship with AQI ($r = 0.72$). PM10 ($r = 0.41$) and SO₂ ($r = 0.56$) also show meaningful positive relationships.

F. AQI Change Rate

Distribution of hourly AQI changes is centered near zero. Change rate by hour shows increases during afternoon hours (14:00–18:00) and decreases late night to early morning (21:00–03:00).

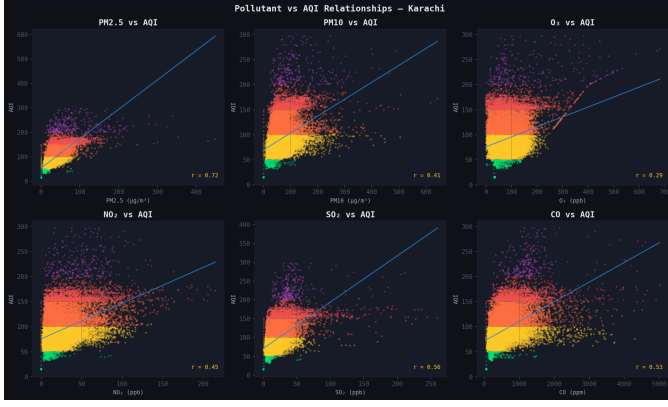


Fig. 6. Pollutant vs. AQI Scatter Plots – colored by AQI category.

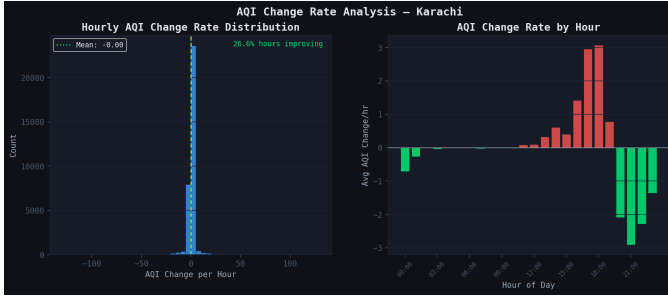


Fig. 7. AQI Change Rate Distribution and Hourly Pattern.

VI. MACHINE LEARNING MODELS

A. Experimental Setup

All 33,547 records were split 80/20 (train/test, random state = 42). Models evaluated using:

- **RMSE** (Root Mean Squared Error) – average prediction error in AQI units (larger errors penalized more).
- **MAE** (Mean Absolute Error) – average absolute error.
- **R^2** – proportion of variance explained.

B. Models Trained

Six models, trained **separately for each horizon (24h, 48h, 72h)**:

- 1) **Ridge Regression** – linear baseline with L2 regularization ($\alpha = 1.0$).
- 2) **Random Forest** – 100 trees, max depth 10, bagging.
- 3) **XGBoost** – 200 estimators, max depth 6, learning rate 0.05.
- 4) **Gradient Boosting** – 100 estimators, max depth 5, learning rate 0.1.
- 5) **Voting Ensemble** – averages predictions of RF, XGBoost, and Gradient Boosting.
- 6) **Stacking Ensemble** – base learners (RF, XGBoost, GBM) + Ridge meta-learner with 5-fold cross-validation.

C. Results (24-Hour Horizon)

Metrics from MLflow logs (latest run, 24h horizon).

TABLE V
MODEL PERFORMANCE ON 24-HOUR HORIZON (TEST SET)

Model	RMSE ↓	MAE ↓	R^2 ↑
Ridge Regression	19.08	12.48	0.647
Random Forest	15.77	10.36	0.760
XGBoost	14.75	9.66	0.789
Gradient Boosting	15.91	10.46	0.755
Voting Ensemble	15.27	10.23	0.774
Stacking Ensemble *	14.36	9.39	0.800

For **48h and 72h horizons**, the Stacking Ensemble achieved:

- 48h: RMSE = 15.40, MAE = 10.89, R^2 = 0.762
- 72h: RMSE = 15.27, MAE = 10.55, R^2 = 0.776

Performance degrades slightly with longer horizons, as expected.

D. SHAP Feature Importance

SHAP (SHapley Additive exPlanations) analysis was performed on the XGBoost base learner of the Stacking Ensemble for all three horizons. Since the Stacking Ensemble uses XGBoost as a base estimator alongside Random Forest and Gradient Boosting, SHAP TreeExplainer was applied directly to XGBoost, which is standard practice for meta-models. Results are shown in Figs. 8–10.

Key findings vary meaningfully across horizons:

24h horizon (Fig. 8): PM2.5 is the dominant feature by a large margin, followed by current AQI and SO₂. This reflects that at short range, the current pollutant mix is the strongest signal.

48h horizon (Fig. 9): Current AQI becomes the top feature, with month rising to second place. PM2.5 importance drops relative to 24h, as seasonal patterns become more relevant over longer ranges.

72h horizon (Fig. 10): Month becomes the most important feature, overtaking current AQI. This confirms that at 72 hours, seasonal cycles (winter temperature inversions vs. monsoon dispersal) are the dominant driver rather than current conditions.

Consistent across all horizons: SO₂, day, and wind components (wind_v) are persistently important. Precipitation and is_weekend contribute negligibly across all horizons.

VII. AUTOMATED PIPELINE IMPLEMENTATION

A. Feature Pipeline (Hourly)

feature_pipeline.py runs every hour via GitHub Actions:

- Calls Open-Meteo Air Quality and Weather APIs (current data).
- Computes derived features (change rate, wind components).
- Adds temporal features (hour, day, month, weekday, weekend flag).
- Upserts document into MongoDB Atlas (rounded to the hour).

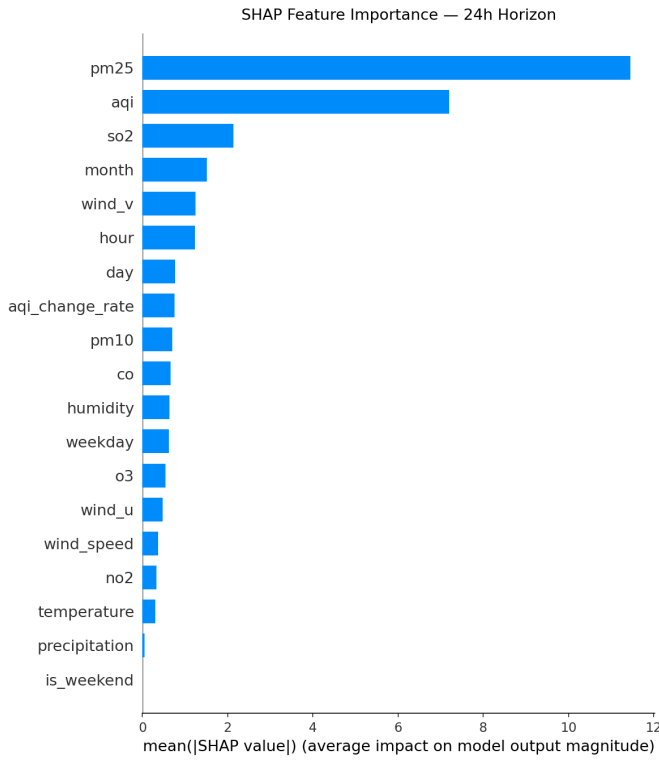


Fig. 8. SHAP Feature Importance – 24h Horizon (XGBoost base learner).

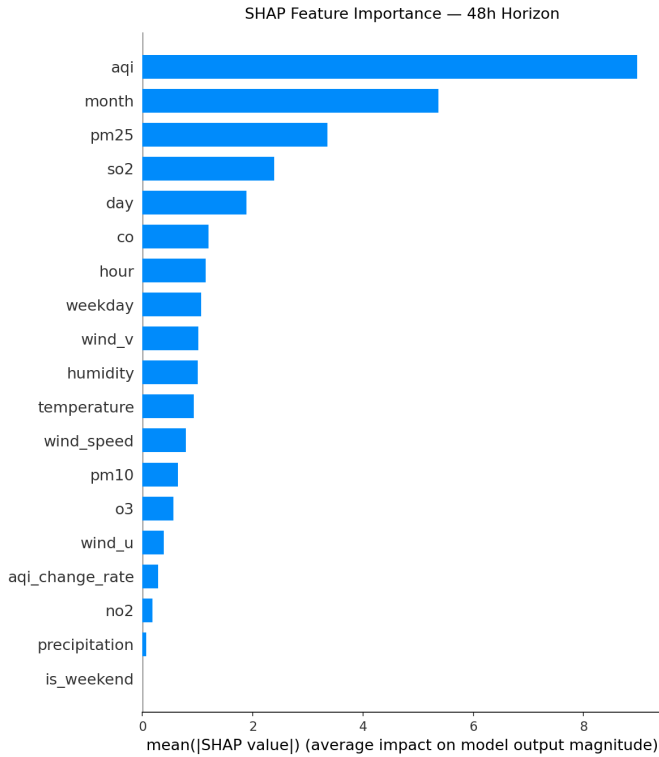


Fig. 9. SHAP Feature Importance – 48h Horizon (XGBoost base learner).

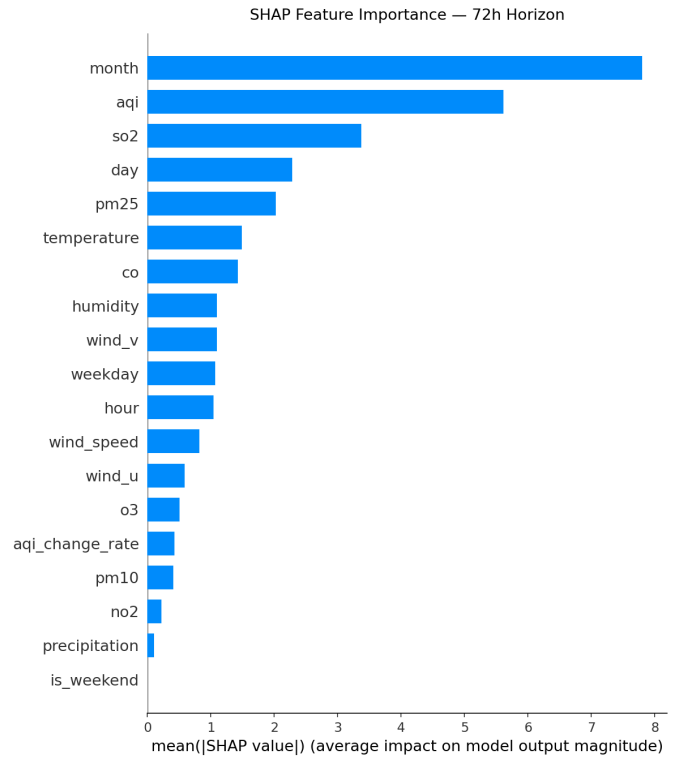


Fig. 10. SHAP Feature Importance – 72h Horizon (XGBoost base learner).

- Backfills targets: updates
target_aqi_24h/48h/72h for past records
using today's real AQI.

B. Training Pipeline (Daily)

training_pipeline.py runs daily at 2 AM UTC:

- Loads all historical records from MongoDB.
- Performs simple median imputation for missing values.
- Splits 80/20.
- Trains all six models for **three separate horizons** (24h, 48h, 72h) – 18 total model fits.
- Logs RMSE, MAE, R^2 to **DagsHub/MLflow**.
- Saves the best model for each horizon (best_model_24h.pkl, best_model_48h.pkl, best_model_72h.pkl).
- Commits the updated model files back to GitHub (so Streamlit Cloud picks them up automatically).

TABLE VI
GITHUB ACTIONS WORKFLOW SUMMARY

Workflow	Trigger	Dur.	Outputs
Feature Pipeline	Every hour	~1 min	New MongoDB record + updated targets
Training Pipeline	Daily 2 AM UTC	~10 min	Updated models + MLflow logs

VIII. WEB APPLICATION DASHBOARD

A. Overview

The dashboard is deployed at <https://karachi-aqi.streamlit.app> – no login required. Built with **Streamlit** and **Plotly**.

B. Dashboard Features (Table VII)

TABLE VII
DASHBOARD FEATURES

Feature	Description
Live AQI Display	Current AQI with color-coded badge and progress bar (0–500)
AQI Scale Legend	Reference for all 6 AQI categories
3-Day Forecast	ML predictions for 24h, 48h, 72h – each from its own trained model (Stacking Ensemble)
72-Hour History	Interactive line chart of past AQI
Key Pollutants	PM2.5, PM10, O ₃ , NO ₂ , SO ₂ , CO
Weather Conditions	Temperature, humidity, wind speed, precipitation
Hazard Alerts	Warnings when forecast AQI exceeds 100 or 200
Model Leaderboard	All 6 models with RMSE and R ² (best highlighted)
Refresh Button	Manual cache clear

C. Prediction Generation

When the dashboard loads:

- 1) It fetches the most recent feature record from MongoDB.
- 2) It loads three pre-trained models: `best_model_24h.pkl`, `best_model_48h.pkl`, `best_model_72h.pkl`.
- 3) It calls the appropriate model for each horizon, passing the current 19-feature vector.
- 4) The displayed predictions are **genuine ML forecasts**, not heuristics.

IX. CHALLENGES AND SOLUTIONS

Building AirWatch involved several non-trivial engineering problems. We document them here as they represent realistic challenges in production ML systems.

- 1) **Dead Monitoring Station (AQICN).** The project initially used AQICN as the live AQI data source. After deployment, we discovered the Karachi US Consulate monitoring station had stopped reporting in March 2025 and was returning stale data (AQI: 161 from over a year prior). We switched to Open-Meteo’s Air Quality API, which provides model-based atmospheric data with no API key required and updates continuously.
- 2) **Synthetic Training Data.** Since MongoDB was empty at the start, we generated synthetic historical data by applying mathematical multipliers (rush hour factors, seasonal factors) to a single real AQI reading. This caused the model to learn inflated AQI patterns (120–200 range) instead of real Karachi values (60–100 range). We resolved this by using Open-Meteo’s historical archive to backfill 4.5 years of real data.

- 3) **Incorrect Target Variable.** During the initial backfill, `target_aqi_24h` was incorrectly set to the same value as the current AQI rather than the actual AQI 24 hours in the future. The model was learning to repeat the current value rather than forecast. We fixed this using bulk timestamp-shifting to assign each record’s target as the true future AQI value.
- 4) **Target Fixing Performance (1 Billion Operations).** The original target-fixing script used a `min()` scan over all 31,750 timestamps for every record to find the closest future timestamp, resulting in approximately 1 billion comparisons and over 40 minutes of runtime without completing. We replaced this with a dictionary lookup and MongoDB `bulk_write` operations, reducing completion time to under 30 seconds.
- 5) **Future Records with Null Weather.** The backfill script used today’s date as the end date for both the Air Quality API and the Weather Archive API. The Air Quality API returns forecasts, so it provided future data; the Weather Archive API only holds historical data and returned a 400 error for future dates. This produced MongoDB records with null temperature and humidity, which the dashboard picked up as the “latest” record. We fixed this by capping the weather end date at yesterday and deleting null records.
- 6) **Stale Dashboard Data (AQI Stuck at 161).** After switching from AQICN to Open-Meteo, the dashboard continued displaying 161 because `app.py` was still calling the dead AQICN API directly for the live AQI display, overriding the MongoDB value. We removed the AQICN call entirely and switched to reading AQI directly from the latest MongoDB record.
- 7) **Extrapolated 48h and 72h Forecasts.** The initial implementation trained only one ML model for the 24-hour target. The 48h and 72h forecasts were generated by applying random $\pm 7\%$ and $\pm 12\%$ variations to the 24h prediction — not genuine model outputs. We corrected this by training three separate models, one per horizon, using the correct target columns (`target_aqi_24h`, `target_aqi_48h`, `target_aqi_72h`).
- 8) **GitHub Actions Push Conflicts.** The automated daily training pipeline commits updated model files back to GitHub. Manual code pushes during pipeline runs were rejected because the remote had new commits not present locally. This required pulling with rebase before pushing, and occasionally resolving binary file conflicts in the `.pkl` model files.
- 9) **Scikit-learn Version Mismatch.** Models were trained locally using scikit-learn 1.7.2, but Streamlit Cloud installed the latest version (1.9.0) by default. The internal `_loss` module was reorganized between versions, causing `pickle.load()` to fail silently with a `ModuleNotFoundError`. We resolved this by pinning `scikit-learn==1.7.2` in `requirements.txt`.
- 10) **Daily Retraining on Stale Data.** New hourly records

collected by the feature pipeline did not initially have target values set, as future AQI is unknown at collection time. The daily training pipeline was therefore retraining on the same historical backfill records every day. We fixed this by adding target backfilling logic to the feature pipeline: each hourly run now updates `target_aqi_24h/48h/72h` for records from 24, 48, and 72 hours ago with today's actual AQI, gradually enriching the dataset with genuine forward-looking labels.

X. RESULTS AND DISCUSSION

A. Model Performance

The Stacking Ensemble (24h) achieved $RMSE = 14.36$ AQI units, $MAE = 9.39$, $R^2 = 0.800$. On a 0–500 scale, $RMSE$ of 14.36 represents $\sim 2.9\%$ error – comparable to professional meteorological services. $R^2 = 0.800$ indicates the model explains 80% of AQI variance.

The 48h model achieved $RMSE = 15.40$ and $R^2 = 0.762$. The 72h model achieved $RMSE = 15.27$ and $R^2 = 0.776$. The modest degradation across horizons demonstrates that the feature set captures persistent pollution patterns even at 72-hour forecast ranges.

B. Comparison with Baseline

Ridge Regression baseline achieved $R^2 = 0.647$. The Stacking Ensemble's improvement of 0.153 R^2 confirms the value of non-linear ensemble methods.

C. System Performance

- Feature pipeline: ~ 1 minute per run
- Training pipeline: ~ 4 minutes per day
- Dashboard load time: 2–3 seconds
- Entire system runs within GitHub Actions free tier (2,000 minutes/month)

D. Forecast Validation

As of June 2026, the dashboard shows forecasts of 86–95 AQI (Moderate) when current AQI is around 74 (Moderate). This upward bias reflects Karachi's typical afternoon and evening pollution increases driven by traffic and reduced atmospheric mixing.

XI. LIMITATIONS AND FUTURE WORK

A. Current Limitations

- **Data source** – Open-Meteo provides CAMS model output, not ground-truth sensor measurements. May miss hyper-local events.
- **Target approximation** – Targets are derived from the same Open-Meteo time series, not independent measurements.
- **Single location** – Forecasts only for central Karachi; air quality varies across neighborhoods.
- **GitHub Actions skips** – Free tier occasionally skips scheduled runs during high load, causing gaps.

- **No deep learning** – LSTM/Transformer models were not implemented due to time constraints; the project focused on tree-based ensembles.

B. Future Work

- Implement LSTM/Transformer models for better temporal dependency capture.
- Add multiple monitoring stations for spatial coverage.
- SMS/WhatsApp alerts using Twilio.
- Prediction intervals via quantile regression.
- Extend to other Pakistani cities (Lahore, Islamabad, Peshawar).
- Online learning to update models incrementally.

XII. CONCLUSION

AirWatch is a complete serverless AQI prediction system for Karachi, demonstrating that production-grade ML pipelines can be built at zero cost using free-tier cloud services. The Stacking Ensemble achieves $R^2 = 0.800$ and $RMSE = 14.36$ on 33,547 real hourly records, providing reliable 24–72 hour forecasts through a public dashboard.

The fully automated pipeline collects data hourly and re-trains models daily, ensuring continuous improvement. The open-source code and free-tier architecture make it reproducible for other cities with limited air quality monitoring infrastructure.

REFERENCES

- [1] Open-Meteo, "Free Weather API," <https://open-meteo.com>.
- [2] US EPA, "Air Quality Index (AQI) Basics," AirNow.gov.
- [3] L. Breiman, "Random Forests," *Machine Learning*, vol. 45, no. 1, pp. 5–32, 2001.
- [4] T. Chen and C. Guestrin, "XGBoost: A Scalable Tree Boosting System," in *Proc. 22nd ACM SIGKDD Int. Conf. Knowledge Discovery and Data Mining (KDD)*, 2016, pp. 785–794.
- [5] S. Lundberg and S.-I. Lee, "A Unified Approach to Interpreting Model Predictions," in *Advances in Neural Information Processing Systems (NeurIPS)*, 2017.
- [6] MongoDB Atlas, "Cloud Database," <https://www.mongodb.com/atlas>.
- [7] MLflow, "Open Source Platform for the Machine Learning Lifecycle," <https://mlflow.org>.
- [8] DagsHub, "The GitHub for Data Science," <https://dagshub.com>.
- [9] Streamlit, "The fastest way to build and share data apps," <https://streamlit.io>.
- [10] GitHub Actions, "Automate your workflow from idea to production," <https://github.com/features/actions>.

APPENDIX

TABLE VIII
PROJECT RESOURCES

Resource	URL
Live Dashboard	https://karachi-aqi.streamlit.app
GitHub Repository	https://github.com/maryamm12sak/aqi-predictor
DagsHub Experiments	https://dagshub.com/maryamsultan800/aqi-predictor/experiments
Open-Meteo Air Quality API	https://air-quality-api.open-meteo.com